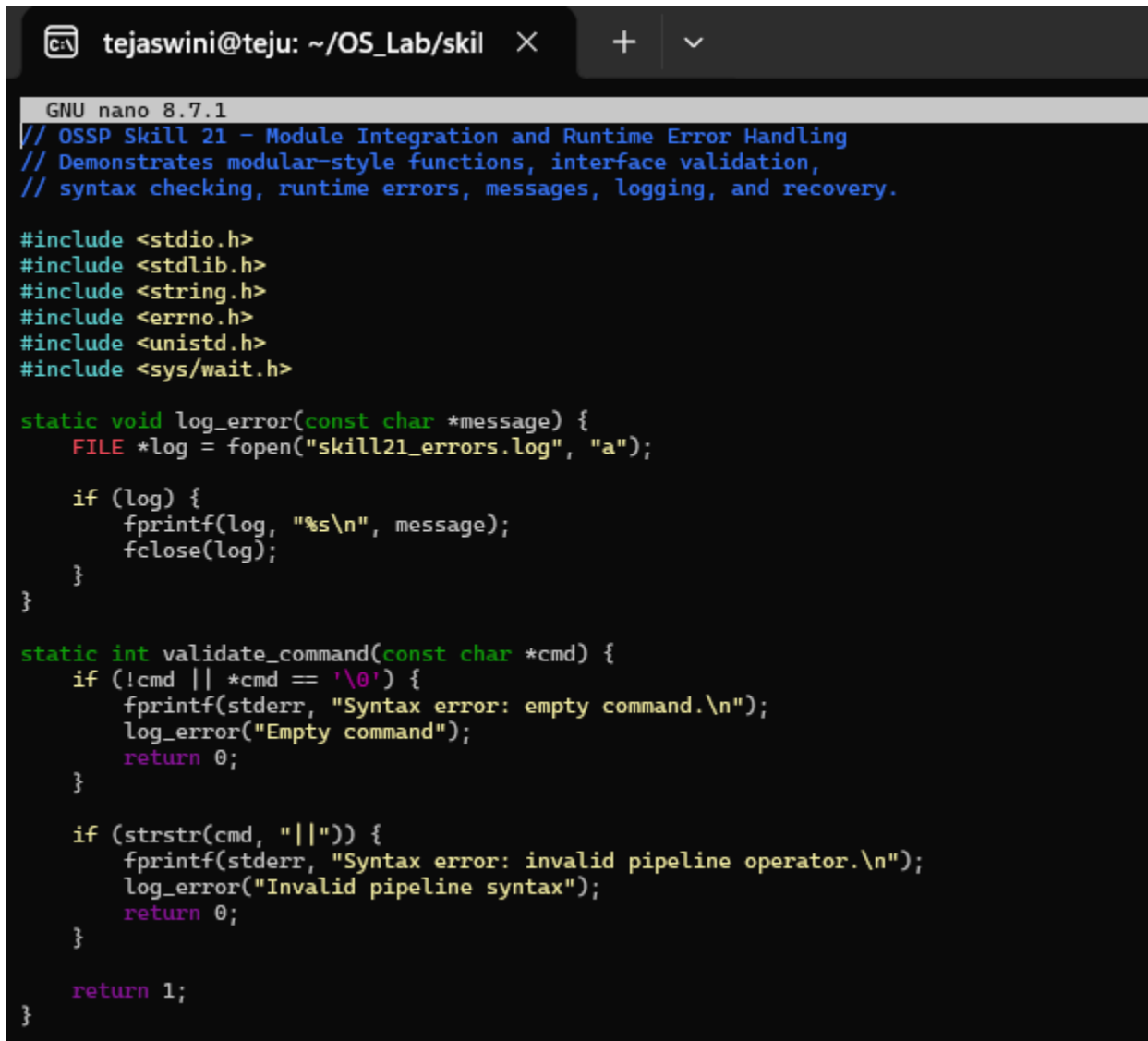


Skill-21

Rollno:2520030011

Program:



```
tejaswini@teju: ~/OS_Lab/skil × + v
GNU nano 8.7.1
// OSSP Skill 21 - Module Integration and Runtime Error Handling
// Demonstrates modular-style functions, interface validation,
// syntax checking, runtime errors, messages, logging, and recovery.

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <unistd.h>
#include <sys/wait.h>

static void log_error(const char *message) {
    FILE *log = fopen("skill21_errors.log", "a");

    if (log) {
        fprintf(log, "%s\n", message);
        fclose(log);
    }
}

static int validate_command(const char *cmd) {
    if (!cmd || *cmd == '\0') {
        fprintf(stderr, "Syntax error: empty command.\n");
        log_error("Empty command");
        return 0;
    }

    if (strstr(cmd, "|")) {
        fprintf(stderr, "Syntax error: invalid pipeline operator.\n");
        log_error("Invalid pipeline syntax");
        return 0;
    }

    return 1;
}
```

```

static int run_command(const char *cmd) {
    pid_t pid = fork();

    if (pid == -1) {
        perror("fork");
        log_error("fork failed");
        return -1;
    }

    if (pid == 0) {
        execl("/bin/sh", "sh", "-c", cmd, (char *)NULL);
        perror("exec");
        _exit(127);
    }

    int status;
    if (waitpid(pid, &status, 0) == -1) {
        perror("waitpid");
        log_error("waitpid failed");
        return -1;
    }

    if (WIFEXITED(status) && WEXITSTATUS(status) != 0) {
        char msg[256];
        snprintf(msg, sizeof(msg), "Command failed with status %d: %s",
                 WEXITSTATUS(status), cmd);
        fprintf(stderr, "%s\n", msg);
        log_error(msg);
    }

    return 0;
}

int main(void) {
    char line[512];

    printf("=== Skill 21: Integration + Error Handling ===\n");
    printf("Try: echo Hello | wc -c\n");
    printf("Try an invalid command to see recovery.\n");
    printf("Type 'exit' to quit.\n\n");

    while (1) {
        printf("integrated> ");

        if (!fgets(line, sizeof(line), stdin))
            break;

        line[strcspn(line, "\n")] = '\0';

        if (strcmp(line, "exit") == 0)
            break;

        if (!validate_command(line))
            continue;

        run_command(line);
    }

    printf("Program ended gracefully. Check skill21_errors.log for failures.\n");
    return 0;
}

```

Output:

```
tojasmin@tpj:~/05_lab/skill1$ nano skill_21.c.c
tojasmin@tpj:~/05_lab/skill1$ gcc skill_21.c.c -o skill_21
tojasmin@tpj:~/05_lab/skill1 $./skill_21
=== Skill 21: Integration + Error Handling ===
try: echo Hello | wc -c
try an invalid command to see recovery.
Type 'exit' to quit.
integrated> echo Hello | wc -c
6
integrated> ls
child_process  escape_sequence.c  prog1.c.save  skill11_input.txt  skill14_error.txt  skill_10.c.c  skill_12  skill_13.c.c  skill_15  skill_16.c.c  skill_18  skill_19.c.c  skill_21  skill_23.c.c  skill_6.c  skill_7  skill_8.c.c
child_process.c  prog1  skill12_output.txt  skill15_combined.txt  skill_11  skill_12.c.c  skill_14  skill_15.c.c  skill_17  skill_18.c.c  skill_20  skill_21.c.c  skill_24.c.c  skill_5  skill_7.c  skill_9
escape_sequence  prog1.c  skill19_append.txt  skill_10  skill_11.c.c  skill_13  skill_14.c.c  skill_16  skill_17.c.c  skill_19  skill_20.c.c  skill_22.c.c  skill_5  skill_6.c  skill_8  skill_9.c.c
integrated> ps
  PID  TTY          TIME CMD
  127 pts/0    00:00:00 bash
  1528 pts/0    00:00:00 skill_21
  1526 pts/0    00:00:00 sh
  1527 pts/0    00:00:00 ps
integrated> tr
tr: missing operand
try 'tr --help' for more information.
Command failed with status 1: tr
integrated> help
sh: 1: help: Permission denied
Command failed with status 127: help
integrated> exit
Program ended gracefully. Check skill21_errors.log for failures.
tojasmin@tpj:~/05_lab/skill1$
```